

JAVA OOP PRACTICAL RECORD

Experiment No. 07

Experiment No.	07
Date	23/09/2026
Title	Method Overriding – Bank Interest
Concept	Inheritance, Method Overriding and Dynamic Polymorphism

1. Objective / Aim

To write a Java program that demonstrates method overriding and dynamic polymorphism by calculating different rates of interest for SBI, ICICI, PNB, and the RBI base class.

2. System Requirements

Hardware: Standard PC/Laptop with 4 GB/8 GB RAM and a Core i3/i5 processor or equivalent.

Software: Windows/Linux/macOS, JDK 17 or JDK 21, and a Java editor or IDE such as VS Code, Eclipse, IntelliJ IDEA, or NetBeans.

3. Algorithm / Logic Description

1. Create Customer and Account classes with their attributes and constructors.
2. Create RBI as the base class with getRateOfInterest() returning 4.0%.
3. Create SBI, ICICI, and PNB classes by extending RBI.
4. Override getRateOfInterest() in each derived bank class with its respective rate.
5. Read the bank name from the user using Scanner.
6. Declare an RBI reference variable named rbi.
7. Create the appropriate derived-class object based on the entered bank name.
8. Use the RBI reference to call getRateOfInterest(); Java selects the overridden method at runtime.
9. Display the bank name and its rate of interest.
10. Close the Scanner.

4. UML Class Diagram

Customer	Account	RBI
- name : String - customerId : int	- accountNumber : int - balance : double	+ getRateOfInterest() : double
+ Customer(name, customerId)	+ Account(accountNumber, balance)	Base Class

SBI	ICICI	PNB
+ getRateOfInterest() : double [override]	+ getRateOfInterest() : double [override]	+ getRateOfInterest() : double [override]

SBI, ICICI and PNB inherit from RBI and override getRateOfInterest().

MethodOverridingBank
+ main(args : String[]) : void

5. Source Code

```
import java.util.Scanner;

// Customer class
class Customer {
    String name;
    int customerId;

    Customer(String name, int customerId) {
        this.name = name;
        this.customerId = customerId;
    }
}

// Account class
class Account {
    int accountNumber;
    double balance;

    Account(int accountNumber, double balance) {
        this.accountNumber = accountNumber;
        this.balance = balance;
    }
}

// RBI - Base Class
class RBI {
    double getRateOfInterest() {
        return 4.0;
    }
}

// SBI - Derived Class
class SBI extends RBI {
    @Override
    double getRateOfInterest() {
        return 7.0;
    }
}

// ICICI - Derived Class
class ICICI extends RBI {
    @Override
    double getRateOfInterest() {
        return 6.5;
    }
}

// PNB - Derived Class
class PNB extends RBI {
    @Override
    double getRateOfInterest() {
        return 6.0;
    }
}

// Main class
public class MethodOverridingBank {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter the Bank name to find the rate of Interest : ");
        String bank = sc.nextLine();

        // Dynamic polymorphism
        RBI rbi;

        if (bank.equalsIgnoreCase("SBI")) {
            rbi = new SBI();
        }
        else if (bank.equalsIgnoreCase("ICICI")) {
            rbi = new ICICI();
        }
        else if (bank.equalsIgnoreCase("PNB")) {
            rbi = new PNB();
        }
        else {
            rbi = new RBI();
        }
        System.out.println(bank + " rate of interest is : "
```

```
        + rbi.getRateOfInterest() + "%");  
    }  
    sc.close();  
}
```

6. Compilation & Execution Commands

```
javac MethodOverridingBank.java  
java MethodOverridingBank
```

7. Sample Output Screen

```
Enter the Bank name to find the rate of Interest : SBI  
SBI rate of interest is : 7.0%  
  
Enter the Bank name to find the rate of Interest : ICICI  
ICICI rate of interest is : 6.5%
```

8. Result

The MethodOverridingBank program was successfully implemented. It demonstrates inheritance, method overriding, and dynamic polymorphism by calling the appropriate bank's getRateOfInterest() method at runtime.